9.2 The term query

A `term` query fetches documents that exactly match a given field. The field is not analyzed. Instead, it is matched against the value stored as is in the inverted index during indexing. For example, using our movie dataset, we can develop a `term` query to search for R-rated movies.

Listing 9.1 Fetching movies with a given rating

```
GET movies/_search
{
  "query": {
    "term": {
      "certificate": "R"
    }
  }
}
```

The term query declaration

The name of the query (`term` in this case) identifies that we are about to perform a term-level search. The object expects the field (`certificate`, in this case) and the search value. Keep in mind that the certificate is a `keyword` data type, so during the indexing process, the value "R" was not processed by any analyzer (actually, it's a `keyword` analyzer which doesn't alter the case); hence it's stored as is.

If you run this query, you'd get all R-rated movies (14 in our sample data set are R-rated). These are wrapped up in the return JSON response. In the next section, we observe the effect of running a term-level search on `text` fields (instead of keyword types).

9.2.1 The term query on text fields

Let's see what happens if we change the query with the rating value to "r" from "R" by lowercasing our search criteria (such as `"certificate": "r"`). Surprisingly, this query doesn't get any results. Can you guess the reason?

Recall from chapter 7 that Elasticsearch analyzes `text` fields during indexing and when searching. The `certificate` field is a `keyword` type, so the field never goes through the analysis process. This means it will always be matched with the contents of the inverted index. When indexing the document, the `certificate` value `"R"` is never tokenized or passed through filters; it's inserted into the inverted index as is.

The other side of the coin is searching: `term` queries are also not analyzed. Unlike a `standard` tokenizer that tokenizes the query's field into multiple tokens and lowercases them, the query field remains as is. If we search for "R", it is considered uppercase because no lowercase conversion (via the `standard` tokenizer) is applied behind the scenes. Therefore, when we search for a lowercase certificate as ("r", for example), there are no matches ("R" was indexed, not "r"), so there is no result.

This brings up an important point about working with `term` queries: `term` queries are unsuitable when working with `text` fields. Although nothing stops us from using